

```

        '''Create a string representation of the fraction'''
        output = ' Fraction: ' + str(self.numerator) + '/' + \
                str(self.denominator) + '\n' + \
                ' Value: ' + str(self.value) + '\n'
        return output

def __add__(self, oOtherFraction): ❹
    ''' Add two Fraction objects'''
    if not isinstance(oOtherFraction, Fraction):
        raise TypeError('Second value in attempt to add is not a Fraction')
    # Use the math package to find the least common multiple
    newDenominator = math.lcm(self.denominator, oOtherFraction.denominator)

    multiplicationFactor = newDenominator // self.denominator
    equivalentNumerator = self.numerator * multiplicationFactor

    otherMultiplicationFactor = newDenominator // oOtherFraction.denominator
    oOtherFractionEquivalentNumerator =
        oOtherFraction.numerator * otherMultiplicationFactor

    newNumerator = equivalentNumerator + oOtherFractionEquivalentNumerator

    oAddedFraction = Fraction(newNumerator, newDenominator)
    return oAddedFraction

def __eq__(self, oOtherFraction): ❺
    '''Test for equality '''
    if not isinstance(oOtherFraction, Fraction):
        return False # not comparing to a fraction
    if (self.numerator == oOtherFraction.numerator) and \
        (self.denominator == oOtherFraction.denominator):
        return True
    else:
        return False

```

Listing 9-10: The `Fraction` class that implements a number of magic methods

When you create a `Fraction` object, you pass in a numerator and a denominator ❶, and the `__init__()` method immediately calculates the reduced fraction and its floating-point value. At any time, client code can call the `getValue()` method to retrieve that value ❷. Client code can also call